

Abstract: Stock Trading Habit Tracker

DLBDSOOFPP01 – Object-Oriented & Functional Programming with Python — Portfolio Part 3
(Finalization Phase)

Aynur Rzayeva-Karabulut · Matriculation No. UPS10746239 · IU Internationale Hochschule

1. Project Overview and Concept

This project implements a Python backend for a habit-tracking application aimed at stock traders, covering daily and weekly routines such as checking a portfolio, reading market news, keeping a trading journal, reviewing weekly performance, and revisiting risk-management rules. The central concept, a Habit, is modelled as a single object-oriented class that owns both its state (name, description, periodicity, creation date, and a log of completion timestamps) and the behaviour needed to reason about that state: checking off a task, editing its details, detecting whether it has lapsed, and calculating its current and longest streaks. A second module implements all analysis functionality — listing habits, filtering by periodicity, and finding the longest streak overall or for a single habit — using the functional programming paradigm (map and filter), so every analytics function is pure and never mutates the list it receives. A command-line interface ties these modules into an interactive menu, and a JSON-based storage module persists habits between sessions. The same logic is also available as a step-by-step Jupyter notebook, which was used throughout development to build and test each part interactively before it was consolidated into the final .py modules.

2. What Went Well

Splitting the project into five small, single-purpose modules (habit, storage, analytics, fixtures, cli) made it straightforward to test each piece in isolation: the 33 pytest cases in the final version each target one module at a time, with no mocking required. Basing both the streak calculation and the broken-habit check on the same underlying idea — counting backward from today (or the current week) through the set of completion dates — kept the two pieces of logic consistent with each other and made them easy to reason about once written down on paper first. Building the project first inside a Jupyter notebook, one small step at a time, also paid off: being able to run a single cell and immediately see its output made it much easier to catch a mistake in, for example, the weekly-streak calculation before it was buried inside a larger file.

3. Pitfalls and Lessons Learned

The most persistent pitfall was not a logic error at all, but a tooling one: pasting code into Jupyter notebook cells repeatedly produced broken indentation, since the editor's auto-indent would sometimes stack on top of indentation already present in the pasted text. This produced confusing errors such as IndentationError: expected an indented block on lines that looked correct at a glance, and on one occasion left two copies of the same method (edit()) inside the class after a fix was pasted a second time without removing the first attempt. The practical lesson was to treat any indentation error as a sign to re-paste the entire function cleanly rather than patch individual lines, and to select-all within a cell before re-indenting it as one block. A second, smaller lesson came from running pytest for the first time: it must be run from the exact

folder containing `test_habits.py`, and it fails with a confusing "import file mismatch" error if the same test file accidentally exists in two places at once — both are easy to miss the first time, but simple to check for once you know to look.

4. Features I Am Most Proud Of

The feature that adds the most genuine value is the pair of `edit_habit()` and `delete_habit()` flows in the CLI. Both reuse the same "list habits, then pick one by number" selection logic, then diverge: editing lets a user leave any field blank to keep its current value, while deleting requires typing an explicit yes before anything is removed, so neither a stray keypress nor an empty prompt can accidentally destroy data. Alongside this, the 5 predefined demo habits with 28 days of hand-crafted completion data made it possible to see the streak and broken-habit logic behave realistically — some habits disciplined, others with gaps — within seconds of starting the app, rather than having to check off habits by hand for weeks to see anything interesting in the analytics report.

5. Conclusion

The finished tracker satisfies the acceptance criteria set out in the assignment: an object-oriented Habit class, daily and weekly periodicities, five predefined habits with four weeks of example data, JSON persistence, a functional-programming analytics module, a command-line interface covering create, check-off, edit, and delete, and a pytest suite covering all of the above. The project, including installation and usage instructions, is hosted at: <https://github.com/<your-username>/stock-trading-habit-tracker>.